PW SKILLS

Understanding SQL DISTINCT

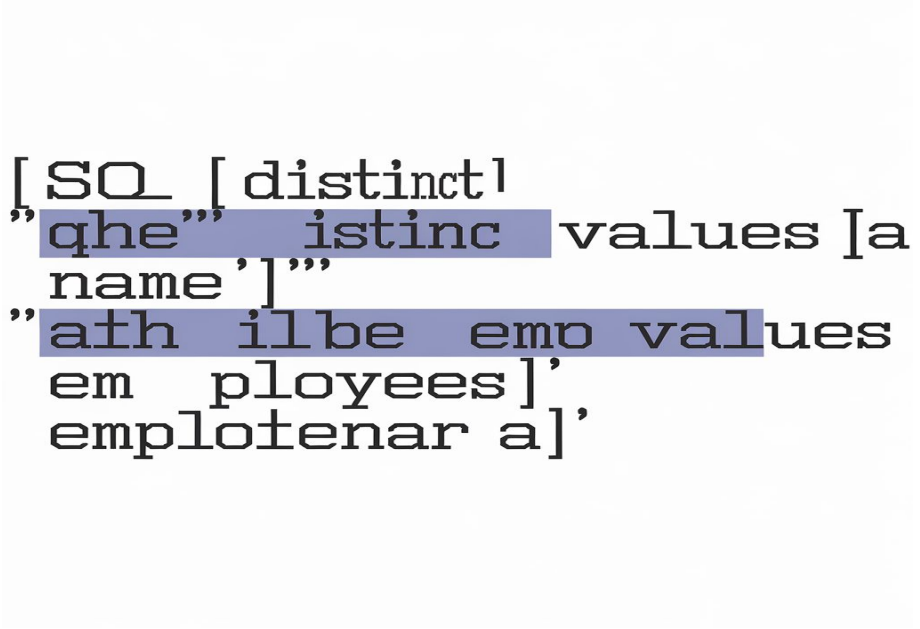
Eliminating Duplicates in SQL Queries



Introduction to SQL DISTINCT

The **DISTINCT keyword in SQL** is used to eliminate duplicate rows from a query result.

It returns only unique records from the selected columns.



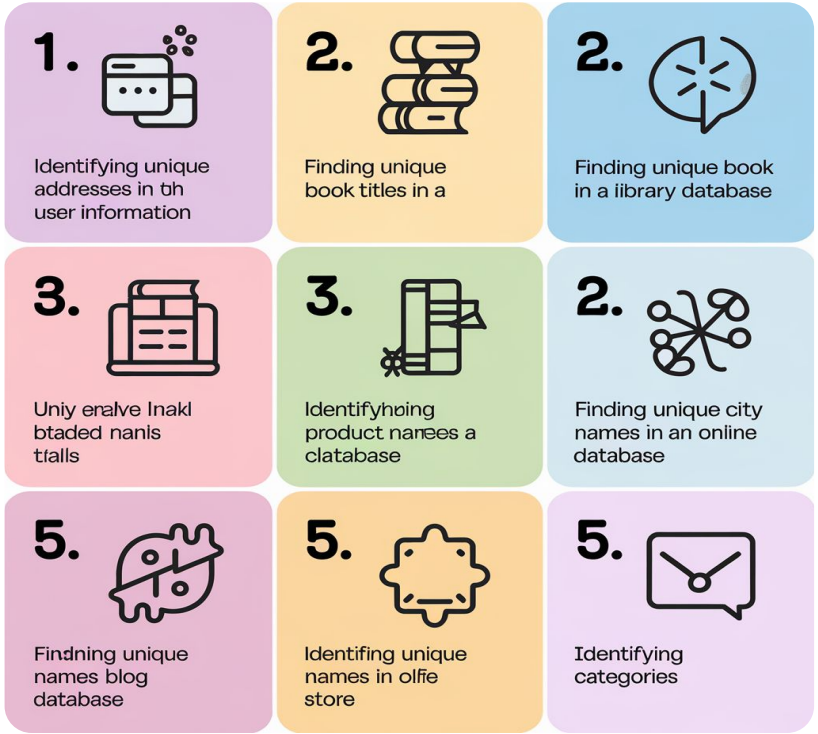
Use Cases for SQL DISTINCT

Removing Duplicate Data: Ensure the uniqueness of data in the results.

Data Summarization: Summarize data by ensuring unique values.

Efficiency: Reduces the number of redundant records processed.

Data Integrity: Helps in cleaning data for analysis or reporting.



Syntax of SQL DISTINCT

For Single Column:

```
SELECT DISTINCT column_name FROM
table_name;

SELECT DISTINCT order_id FROM orders;
```

For Multiple Columns:

```
SELECT DISTINCT column1, column2,
column3 FROM table_name;

SELECT DISTINCT order_id,
product_name, order_date FROM orders;
```

Original Table

order_id	customer_id	product_name	order_date
1	101	Laptop	2025-01-01
2	102	Mouse	2025-01-02
3	101	Keyboard	2025-01-03
4	103	Laptop	2025-01-04
5	101	Laptop	2025-01-05
NULL	NULL	NULL	NULL

Single Column

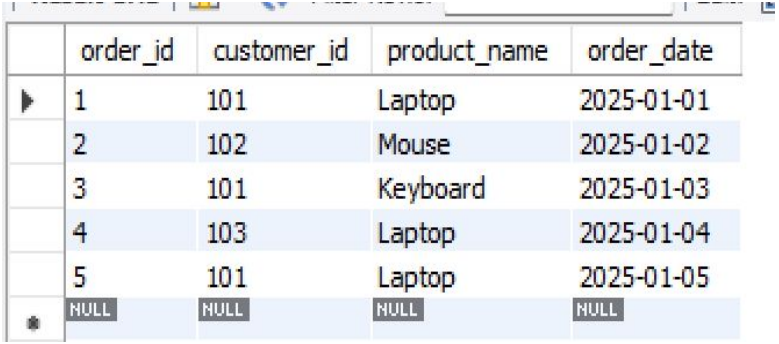
order_id
1
2
3
4
5
NULL

Multiple Columns

order_id	product_name	order_date
1	Laptop	2025-01-01
2	Mouse	2025-01-02
3	Keyboard	2025-01-03
4	Laptop	2025-01-04
5	Laptop	2025-01-05
NULL	NULL	NULL

Counting Distinct Values in SQL

The `COUNT(DISTINCT column_name)` function is used to count the number of unique (distinct) values in a specified column. This is useful when you need to know how many unique entries exist in a dataset for a particular column, such as counting unique products sold or the number of distinct customers.



order_id	customer_id	product_name	order_date
1	101	Laptop	2025-01-01
2	102	Mouse	2025-01-02
3	101	Keyboard	2025-01-03
4	103	Laptop	2025-01-04
5	101	Laptop	2025-01-05
NULL	NULL	NULL	NULL

Syntax:

```
SELECT COUNT(DISTINCT column_name)
FROM table_name;
```

```
SELECT COUNT(DISTINCT product_name)
FROM orders;
```

COUNT(DISTINCT product_name)
3

Key Points to Remember

- **DISTINCT** only works on the columns specified in the query.
- It's case-sensitive, meaning "Laptop" and "laptop" would be considered distinct.
- Using DISTINCT on multiple columns will return unique combinations of values from those columns.
- Can be used with aggregate functions like COUNT.

Best Practices and Performance Considerations

- **Avoid Overusing DISTINCT:** Using DISTINCT on large datasets can slow down performance.
- **Indexes:** Ensure your tables are indexed to optimize queries that use DISTINCT.
- **Use with Care:** Only apply DISTINCT when necessary to maintain performance and data accuracy.

Conclusion

- The DISTINCT keyword is a powerful tool for removing duplicates and summarizing data in SQL queries.
- It is crucial for maintaining clean data and producing accurate results.
- When used effectively, it enhances the readability and utility of the data.

Quiz

Question: What will the following SQL query return?

SELECT COUNT(DISTINCT product_name)

FROM orders;

A

The total number of rows in the orders table

B

The total number of distinct customer IDs

C

The number of unique product names in the orders table

D

number of The total products ordered

Answer & Explanation

A

The total number of rows in the orders table

B

The total number of distinct customer IDs

C

The number of unique product names in the orders table

D

The number of total products ordered

Explanation:

The query uses the COUNT(DISTINCT product_name) function, which counts the number of unique (distinct) product names in the orders table. It ignores duplicate entries for the same product. In the example dataset, even if "Laptop" appears multiple times, it will be counted only once.

The other options are incorrect because:

- Option 1 counts rows, not distinct values.
- Option 2 counts distinct customer IDs, not products.
- Option 4 would count the total number of products, including duplicates.

Sorting and Limiting Results with ORDER BY and LIMIT in SQL

The ORDER BY and LIMIT clauses are useful for controlling the order of query results and restricting the number of rows returned. These features are especially helpful when working with large datasets or when you want to retrieve specific subsets of data.

ORDER BY Clause: Sorting Results

The ORDER BY clause is used to sort the result set of a query. By default, it sorts the data in ascending order. However, you can specify descending order as well.

Syntax:

```
SELECT column1, column2 FROM
table_name ORDER BY column_name [ASC |
DESC];
```

```
SELECT order_id, product_name,
order_date FROM orders ORDER BY
order_date DESC;
```

Original Table

order_id	customer_id	product_name	order_date
1	101	Laptop	2025-01-01
2	102	Mouse	2025-01-02
3	101	Keyboard	2025-01-03
4	103	Laptop	2025-01-04
5	101	Laptop	2025-01-05
NULL	NULL	NULL	NULL

Result

order_id	product_name	order_date
5	Laptop	2025-01-05
4	Laptop	2025-01-04
3	Keyboard	2025-01-03
2	Mouse	2025-01-02
1	Laptop	2025-01-01
NULL	NULL	NULL

LIMIT Clause: Restricting the Number of Rows

The `LIMIT` clause is used to limit the number of rows returned in a result set. This is particularly useful when you need a small subset of data or are working with large datasets where you don't need all the records.

Syntax:

```
SELECT column1, column2
FROM table_name
ORDER BY column_name [ASC | DESC]
LIMIT number_of_rows;
```

Example:

Using the same orders table, if you want to **retrieve only the first 3 orders sorted by order date in descending order**, you can write:

```
SELECT * FROM orders
ORDER BY order_date DESC
LIMIT 3;
```

LIMIT Clause: Restricting the Number of Rows

Original Table

order_id	customer_id	product_name	order_date
1	101	Laptop	2025-01-01
2	102	Mouse	2025-01-02
3	101	Keyboard	2025-01-03
4	103	Laptop	2025-01-04
5	101	Laptop	2025-01-05
NULL	NULL	NULL	NULL

Result

order_id	customer_id	product_name	order_date
5	101	Laptop	2025-01-05
4	103	Laptop	2025-01-04
3	101	Keyboard	2025-01-03
NULL	NULL	NULL	NULL

Key Points:

ORDER BY allows you to sort the result set by one or more columns.

- **ASC** is the default sorting order.
- **DESC** sorts data in descending order.

LIMIT restricts the number of rows returned in the query result.

- This is especially useful in large datasets or when you only need a sample of data.

You can combine **ORDER BY** with **LIMIT** to sort the results and then restrict the number of rows returned, which is commonly used for pagination or fetching top records (like top 5 sales).

Quiz

Question: What does the following SQL query do?

```
SELECT *

FROM orders

ORDER BY order_date DESC

LIMIT 2;
```

A

Retrieves the 2 most recent orders from the **orders** table sorted by **order_date** in descending order.

B

Retrieves all records from the **orders** table sorted by **order_date** in ascending order and limits the result to 2 rows.

C

Retrieves all records from the **orders** table sorted by **order_date** in descending order and limits the result to 2 rows.

D

Retrieves the top 2 products sold in the **orders** table sorted by **order_date** in descending order.

Answer & Explanation

A

Retrieves the 2 most recent `orders` from the orders table sorted by `order_date` in descending order.

B

Retrieves all records from the `orders` table sorted by `order_date` in ascending order and limits the result to 2 rows.

C

Retrieves all records from the `orders` table sorted by `order_date` in descending order and limits the result to 2 rows.

D

Retrieves the top 2 products sold in the `orders` table sorted by `order_date` in descending order.

Explanation:

- **`ORDER BY order_date DESC:`** Sorts the data by the order_date column in descending order (i.e., most recent first).
- **`LIMIT 2:`** Limits the result to the first 2 rows (after sorting).

Combining DISTINCT, LIMIT, and ORDER BY for Precise Data Retrieval in SQL

By combining DISTINCT, ORDER BY, and LIMIT, you can:

- Remove duplicate records using DISTINCT.
- Sort the data in a specific order using ORDER BY.
- Limit the number of rows returned using LIMIT.

Example:

```
SELECT DISTINCT column_name
FROM table_name
ORDER BY column_name DESC
LIMIT number_of_rows;
```

```
SELECT DISTINCT product_name, order_date
FROM orders
ORDER BY order_date DESC
LIMIT 3;
```

Original Table

order_id	customer_id	product_name	order_date
1	101	Laptop	2025-01-01
2	102	Mouse	2025-01-02
3	101	Keyboard	2025-01-03
4	103	Laptop	2025-01-04
5	101	Laptop	2025-01-05
NULL	NULL	NULL	NULL

Result

product_name	order_date
Laptop	2025-01-05
Laptop	2025-01-04
Keyboard	2025-01-03

Use Cases for Combining DISTINCT, ORDER BY, and LIMIT:

Top N Most Recent Products:

- **Use Case:** Retrieve the most recent distinct products ordered.
- **Example Query:** `SELECT DISTINCT product_name FROM orders ORDER BY order_date DESC LIMIT 5;`

Unique Customers in Descending Order of First Order:

- **Use Case:** Get a list of unique customers who have made an order, sorted by the date of their first order.
- **Example Query:** `SELECT DISTINCT customer_id FROM orders ORDER BY order_date DESC LIMIT 10;`

Advantages of Using This Combination:

- **Eliminates Redundancy:** `DISTINCT` helps in removing duplicate records, ensuring that only unique values are returned.
- **Efficient Data Retrieval:** `LIMIT` helps to restrict the result set, making it faster to retrieve smaller sets of data, especially in large datasets.
- **Custom Sorting:** `ORDER` BY allows you to sort the data according to specific criteria, providing more flexibility in the result set.

Quiz

Question: What will the following SQL query return?

```
SELECT DISTINCT product_name

FROM orders

ORDER BY order_date DESC

LIMIT 2;
```

A

The 2 most recent distinct products ordered from the **orders** table.

B

The 2 most recent **orders**, including duplicates, from the `orders` table.

C

The 2 oldest distinct products **ordered** from the **orders** table.

D

The top 2 products based on the total number of **orders**.

Answer

A

The 2 most recent distinct products ordered from the `orders` table.

B

The 2 most recent **orders**, including duplicates, from the orders table.

C

The 2 oldest distinct products ordered from the **orders** table.

D

The top 2 products based on the total number of **orders**.

Explanation:

- `DISTINCT product_name:` This ensures that the query retrieves only unique product names, eliminating duplicates.
- `ORDER BY order_date DESC:` This sorts the results by the order_date in descending order, meaning the most recent orders will appear first.
- `LIMIT 2:` This restricts the query to return only the top 2 products based on the most recent orders.

Introduction to Query Optimization

Why Query Optimization is Important:

- Ensures faster data retrieval, especially for large datasets.
- Reduces server load and improves application performance.
- Helps retrieve only the necessary data, reducing data transfer time.

Performance Tips for Large Datasets

- **Use DISTINCT only when necessary** as it can be computationally expensive.
- **Limit data with LIMIT** to avoid fetching unnecessary rows.
- **Use ORDER BY only if necessary** to avoid additional overhead.
- **Create Indexes** on frequently queried columns for faster searches.

Summary

- **DISTINCT** ensures uniqueness and removes duplicates.
- **ORDER BY** allows for sorting, while **LIMIT** helps restrict the data returned.
- Combining these clauses ensures cleaner, faster, and more efficient queries.
- Follow best practices for better query performance, especially when working with large datasets.

Quiz

Question 1: Which of the following statements is true regarding the use of the `DISTINCT` keyword in SQL?

A

`DISTINCT` will only remove duplicate records when used with an `ORDER BY` clause.

B

`DISTINCT` ensures that only unique values are returned from a specified column or set of columns.

C

`DISTINCT` can only be used with numeric data types to remove duplicates.

D

`DISTINCT` increases query performance by reducing the number of columns returned.

Answer

A

DISTINCT will only remove duplicate records when used with an **ORDER BY** clause.

B

DISTINCT ensures that only unique values are returned from a specified column or set of columns.

C

DISTINCT can only be used with numeric data types to remove duplicates.

D

DISTINCT increases query performance by reducing the number of columns returned.

Explanation:

- The **DISTINCT** keyword is used to eliminate duplicate values from the result set, ensuring that only unique values are returned for the selected columns.
- It works on all data types (not just numeric) and doesn't require the use of ORDER BY.

Practical Use Cases of DISTINCT, LIMIT, and ORDER BY

Clause	Practical Use	Case Example
DISTINCT	Remove duplicate values and ensure uniqueness in data.	Retrieving a list of unique cities from a customer database.
LIMIT	Restrict the number of rows returned, useful for pagination or top-N results.	Showing the top 5 sales transactions based on the highest revenue.
ORDER BY	Sort data in ascending or descending order, useful for ranking and organization.	Sorting products by price to show the cheapest or most expensive items.
Combined	Filter out duplicates, sort data, and limit the result set for efficient retrieval.	Retrieving the top 10 most recent distinct products ordered, sorted by date, and limiting the output to the top 10.

Quiz Question

Question : Which of the following best describes the use of the **DISTINCT** clause in SQL?

A

It ensures that only unique rows are returned by eliminating duplicates from the result set.

B

It sorts the data in ascending or descending order.

C

It limits the number of rows returned by the query.

D

It groups the data into specific categories based on certain conditions.

Answer

A

It ensures that only unique rows are returned by eliminating duplicates from the result set.

B

It sorts the data in ascending or descending order.

C

It limits the number of rows returned by the query.

D

It groups the data into specific categories based on certain conditions.

Explanation:

- `DISTINCT` is used in SQL to eliminate duplicate records from the result set, ensuring that only unique values are returned.

Thanks !

Thank You for attending the Session!!